

Compressing Uncertainty in Artificial Intelligence

Lester Mackey

Microsoft Research New England

August 20, 2026

Joint work with Raaz Dwivedi, Marina Riabiz, Wilson Ye Chen, Jon Cockayne, Pawel Swietach, Steven A. Niederer, Chris J. Oates, Abhishek Shetty, Carles Domingo-Enrich, Lingxiao Li, Annabelle M. Carrell, Albert Gong, and Tobias Schröder

Motivation: Computational Cardiology

Computational Cardiology: Developing multiscale *digital twins* of human hearts to non-invasively predict disease progression and therapy response [Niederer, Sacks, Girolami, and Willcox, 2021]

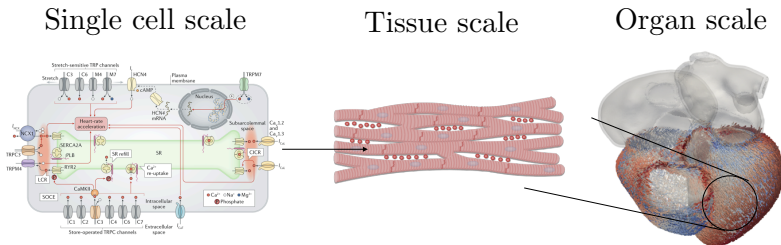


Figure credit:
Marina Riabiz

Example (Heartbeats and arrhythmias)

- Whole-organ heartbeats are coordinated by calcium signaling in heart cells
- Dysregulation known to lead to life-threatening heart arrhythmias
- **Goal:** Model impact of calcium signaling dysregulation on heart function [Campos, Shiferaw, Prassl, Boyle, Vigmond, and Plank, 2015, Niederer, Lumens, and Trayanova, 2019, Colman, 2019]

Motivation: Computational Cardiology

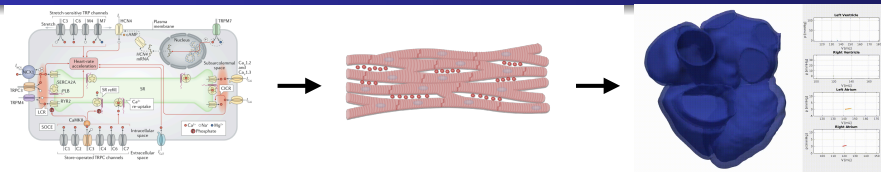


Figure credit:
Augustin
et al.
2020

Inferential Pipeline (Impact of calcium signaling dysregulation on heart function)

- 1 Estimate unknown calcium signaling model parameters from patient data
- 2 Capture uncertainty by sampling many likely parameter configurations
 - Run **Markov chain Monte Carlo (MCMC)** to (eventually) draw sample points from the posterior distribution $\mathbb{P}$ over unknown parameters
 - May require **millions** of sample points to adequately explore target distribution $\mathbb{P}$
- 3 Propagate uncertainty by simulating whole-heart model for each configuration
 - **Problem:** Each simulation requires **thousands of CPU hours!**

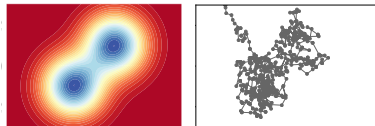
Questions: Can we accurately summarize $\mathbb{P}$ using many fewer points? If so, how?

Distribution Compression

Goal: Accurately summarize a distribution $\mathbb{P}$ using a small number of points

Standard solutions

- **i.i.d. sampling** directly from $\mathbb{P}$
- **MCMC** with Markov chain converging to $\mathbb{P}$



Benefits: Readily available and eventually high-quality

- Provide asymptotically exact sample estimates $\mathbb{P}_n f = \frac{1}{n} \sum_{i=1}^n f(x_i)$ for intractable expectations $\mathbb{P}f = \mathbb{E}_{X \sim \mathbb{P}}[f(X)]$

Drawback: Samples are too large!

- Typical integration error $\mathbb{P}_n f - \mathbb{P}f = \Theta(n^{-1/2})$: need $n = 10000$ for 1% error
- **Prohibitive** for expensive downstream tasks and function evaluations

Idea: Directly compress the high-quality sample approximations $\mathbb{P}_n$

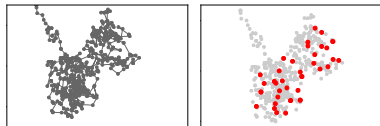
- Reduces general problem to approximating empirical distributions

Distribution Compression

Question: How do we effectively compress an empirical distribution $\mathbb{P}_n$?

Standard solutions

- Uniform subsampling / i.i.d. sampling
- Standard thinning: Keep every t -th point



Drawback: Large loss in accuracy, worst case integration error = $\Theta(\sqrt{t/n})$

- Compression from n to $\sqrt{n}$ points increases error from $\Theta(n^{-1/2})$ to $\Theta(n^{-1/4})$

Question: Can we do better?

Minimax lower bounds for worst-case integration error to $\mathbb{P}$

- $\Omega(n^{-1/2})$ for any compression procedure returning $\sqrt{n}$ points [Phillips and Tai, 2020]
- $\Omega(n^{-1/2})$ for any function of n i.i.d. points from $\mathbb{P}$ [Tolstikhin, Sriperumbudur, and Muandet, 2017]

This talk: Introduce practical compression strategies that match these lower bounds up to log factors, even for **nonuniform** and **unbounded** $\mathbb{P}$

Problem Setup

Given:

- **Input points** $\mathcal{S}_{\text{in}} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ with empirical distribution $\mathbb{P}_n = \frac{1}{n} \sum_{i=1}^n \delta_{x_i}$
 - Pre-generated by any algorithm (i.i.d. sampling, MCMC, quadrature, kernel herding)
- Target output size s (e.g., $s = \sqrt{n}$ for heavy compression)

Goal: Return **coreset** $\mathcal{S}_{\text{out}} \subset \mathcal{S}_{\text{in}}$ with $|\mathcal{S}_{\text{out}}| = s$, $\mathbb{Q} = \frac{1}{s} \sum_{x \in \mathcal{S}_{\text{out}}} \delta_x$, and $o(s^{-1/2})$ (better-than-i.i.d.) worst-case integration error between $\mathbb{P}_n$ and $\mathbb{Q}$

Quality Measures

Goal: Return **coreset** $\mathcal{S}_{\text{out}} \subset \mathcal{S}_{\text{in}}$ with $|\mathcal{S}_{\text{out}}| = s$, $\mathbb{Q} = \frac{1}{s} \sum_{x \in \mathcal{S}_{\text{out}}} \delta_x$, and $o(s^{-1/2})$ worst-case integration error between $\mathbb{P}_n$ and $\mathbb{Q}$

Maximum mean discrepancy (MMD) [Gretton, Borgwardt, Rasch, Schölkopf, and Smola, 2012]

$$\text{MMD}_{\mathbf{k}}(\mathbb{P}_n, \mathbb{Q}) = \sup_{\|f\|_{\mathbf{k}} \leq 1} |\mathbb{P}_n f - \mathbb{Q} f|$$

- Measures **maximum discrepancy between input and coreset expectations** over a class of real-valued test functions (unit ball of a reproducing kernel Hilbert space)
- Parameterized by a **reproducing kernel $\mathbf{k}$** : any **symmetric** ($\mathbf{k}(x, y) = \mathbf{k}(y, x)$) and **positive semidefinite** ($\sum_{i,l} c_i c_l \mathbf{k}(z_i, z_l) \geq 0, \forall z_i \in \mathbb{R}^d, c_i \in \mathbb{R}$) function
 - Gaussian: $\mathbf{k}(x, y) = e^{-\frac{1}{2}\|x-y\|_2^2}$, Inverse multiquadric: $\mathbf{k}(x, y) = \frac{1}{(1+\|x-y\|_2^2)^{1/2}}$

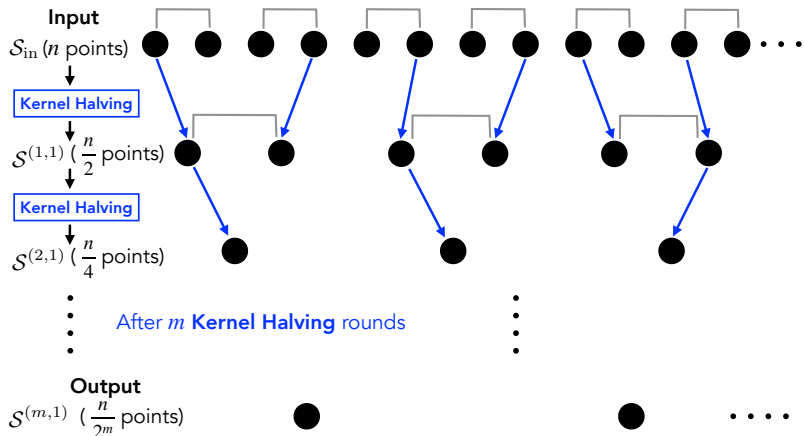
Kernel sub-Gaussian constant

$$\sigma_{\mathbf{k}} = \sup_f \frac{\sqrt{2 \log \mathbb{E}[\exp(\mathbb{P}_n f - \mathbb{Q} f) \mid \mathbb{P}_n]}}{\|f\|_{\mathbf{k}}}$$

$\Rightarrow$ Each $\mathbb{Q}f$ is concentrated around $\mathbb{P}_n f$ with variance $\leq \sigma_{\mathbf{k}}^2 \|f\|_{\mathbf{k}}^2$

1 Initialization: KT-SPLIT

- Partitions input $\mathcal{S}_{\text{in}}$ into balanced candidate coresets, each of size s



Kernel Halving [Dwivedi and Mackey, 2024]

Goal: Split $\mathcal{S}_{\text{in}}$ into two **balanced** coresets $\mathcal{S}_{\text{out}}, \mathcal{S}'_{\text{out}}$ of equal size

- **Balance:** $\mathbb{Q}\mathbf{k} \approx \mathbb{Q}'\mathbf{k} \Leftrightarrow \mathbb{P}_n\mathbf{k} \approx \mathbb{Q}\mathbf{k} \Leftrightarrow \sigma_{\mathbf{k}} \text{ small} \quad \text{for} \quad \mathbb{Q}'\mathbf{k} \triangleq \frac{1}{|\mathcal{S}'_{\text{out}}|} \sum_{x \in \mathcal{S}'_{\text{out}}} \mathbf{k}(x, \cdot)$

Uniformly random halving: $\sigma_{\mathbf{k}} = \Omega(\frac{1}{\sqrt{n}})$

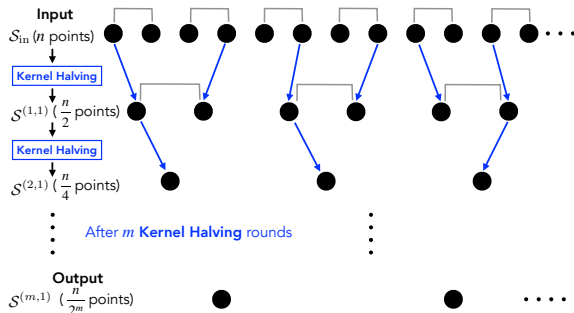
Kernel halving: Use **non-uniform** randomness to encourage balance

- Hilbert space generalization of self-balancing walk of Alweiss, Liu, and Sawhney [2020]
- Start with empty coresets $\mathcal{S}_{\text{out}}, \mathcal{S}'_{\text{out}}$
- Assign input points $(x, x') = (x_1, x_2), \dots, (x_{n-1}, x_n)$ to coresets two at a time:
 - ① Try adding x to $\mathcal{S}_{\text{out}}$ and x' to $\mathcal{S}'_{\text{out}}$ and record $\alpha_{\text{heads}} = \|\mathbb{Q}\mathbf{k} - \mathbb{Q}'\mathbf{k}\|_{\mathbf{k}}$
 - ② Try adding x' to $\mathcal{S}_{\text{out}}$ and x to $\mathcal{S}'_{\text{out}}$ and record $\alpha_{\text{tails}} = \|\mathbb{Q}\mathbf{k} - \mathbb{Q}'\mathbf{k}\|_{\mathbf{k}}$
 - ③ Final assignment: flip coin biased toward the more balanced option (the smaller α)
- Enjoys **dimension-free** $\sigma_{\mathbf{k}} \leq \frac{\sqrt{4 \log(n/\delta) \|\mathbf{k}\|_{\infty}}}{n}$ with probability $1 - \delta$

[Carrell, Gong, Shetty, Dwivedi, and Mackey, 2025]

1 Initialization: KT-SPLIT

- Partitions input $\mathcal{S}_{\text{in}}$ into balanced candidate coresets, each of size s



- Non-uniform randomness ensures $\sigma_{\mathbf{k}} = \mathcal{O}(\frac{\sqrt{\log s}}{s})$ after all halving rounds
- Theorem:** $\text{MMD}_{\mathbf{k}} = \begin{cases} \mathcal{O}(\sigma_{\mathbf{k}} \log^{\frac{a}{2}} s) & \text{for exponential eigendecay } \lambda_{i+1} = \mathcal{O}(ne^{-ci^{1/a}}) \\ o(s^{-\frac{1}{2}}) & \text{for polynomial eigendecay } \lambda_{i+1} = \mathcal{O}(n/i^a), a > 2 \end{cases}$
with high probability vs. $\Omega(s^{-\frac{1}{2}})$ for i.i.d. coreset [Carrell, Gong, Shetty, Dwivedi, and Mackey, 2025]

1 Initialization: KT-SPLIT

- Partitions input $\mathcal{S}_{\text{in}}$ into balanced candidate coresets, each of size s
- Non-uniform randomness ensures $\sigma_{\mathbf{k}} = \mathcal{O}(\frac{\sqrt{\log s}}{s})$ after all halving rounds
- **Thm:** $\text{MMD}_{\mathbf{k}} = \tilde{\mathcal{O}}(s^{-1})$ for exponential eigendecay vs. $\Omega(s^{-\frac{1}{2}})$ for i.i.d. [Dwivedi and Mackey, 2024]

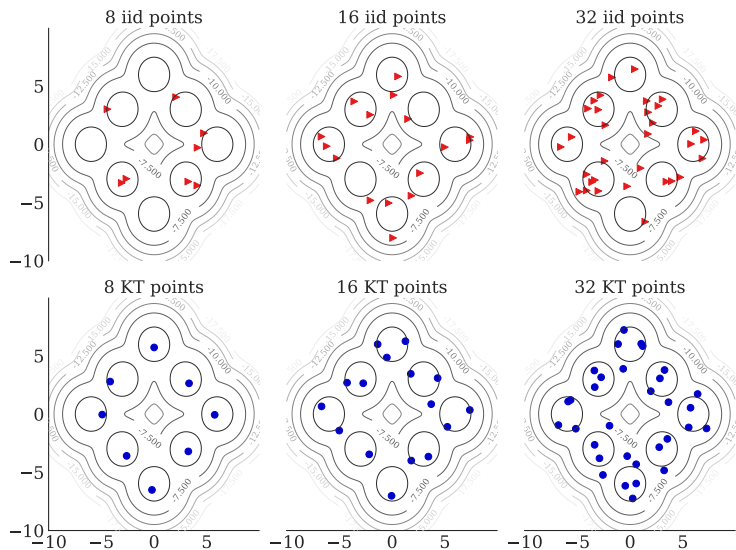
2 Refinement: KT-SWAP

- Selects candidate coreset closest to $\mathcal{S}_{\text{in}}$ in terms of $\text{MMD}_{\mathbf{k}}$
- Iteratively refines the coreset by replacing each coreset point in turn with the best alternative in $\mathcal{S}_{\text{in}}$, as measured by $\text{MMD}_{\mathbf{k}}$

Complexity

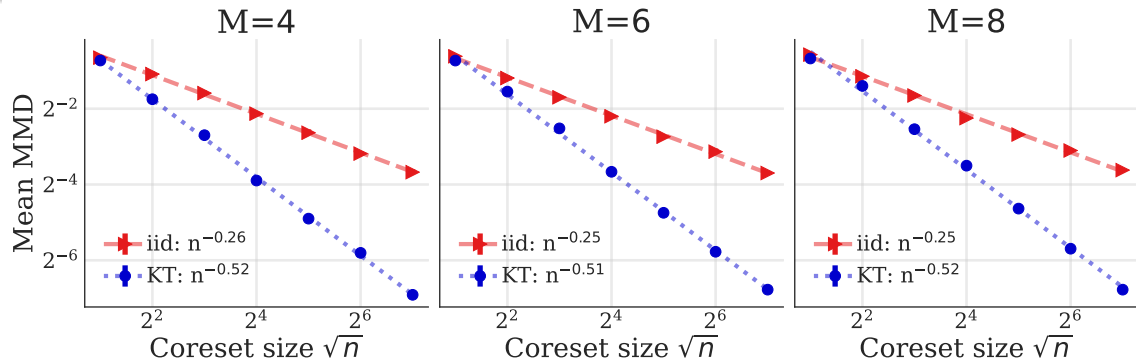
- Time: dominated by $\mathcal{O}(n^2)$ kernel evaluations
 - Reduces to $\mathcal{O}(n \log^3 n)$ for $s = \sqrt{n}$ using **Compress++** of Shetty, Dwivedi, and Mackey [2022]
- Space: $\mathcal{O}(\min(nd, n^2))$
 - Reduces to $\mathcal{O}(\sqrt{nd} \log n)$ for $s = \sqrt{n}$ using **Compress++**

Kernel Thinning vs. i.i.d. Sampling: Mixture of Gaussians



- $\mathbb{P} = \frac{1}{M} \sum_{j=1}^M \mathcal{N}(\mu_j, \mathbf{I}_d)$
- $\mathbf{k}(x, y) = \exp(-\frac{1}{2\sigma^2} \|x - y\|_2^2)$ with $\sigma^2 = 2d$
- Even for small sample sizes, kernel thinning (KT) provides
 - Better stratification across components
 - Less clumping and fewer gaps within components

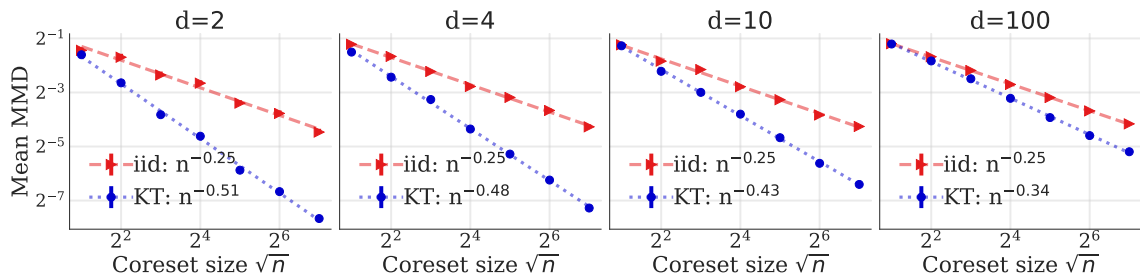
Kernel Thinning vs. i.i.d. Sampling: Mixture of Gaussians



Kernel thinning (KT) improves both rate of decay and order of magnitude of $\text{MMD}_{\mathbf{k}}(\mathbb{P}, \mathbb{Q}_{KT})$

- $\mathbb{P} = \frac{1}{M} \sum_{j=1}^M \mathcal{N}(\mu_j, \mathbf{I}_d)$, $d = 2$
- $\mathbf{k}(x, y) = \exp(-\frac{1}{2\sigma^2} \|x - y\|_2^2)$ with $\sigma^2 = 2d$

Kernel Thinning vs. i.i.d. Sampling: Higher Dimensions



Kernel thinning (KT) improves both rate of decay and order of magnitude of $\text{MMD}_{\mathbf{k}}(\mathbb{P}, \mathbb{Q}_{KT})$ even for high dimensions and small sample sizes

- $\mathbb{P} = \mathcal{N}(0, \mathbf{I}_d)$
- $\mathbf{k}(x, y) = \exp(-\frac{1}{2\sigma^2} \|x - y\|_2^2)$ with $\sigma^2 = 2d$

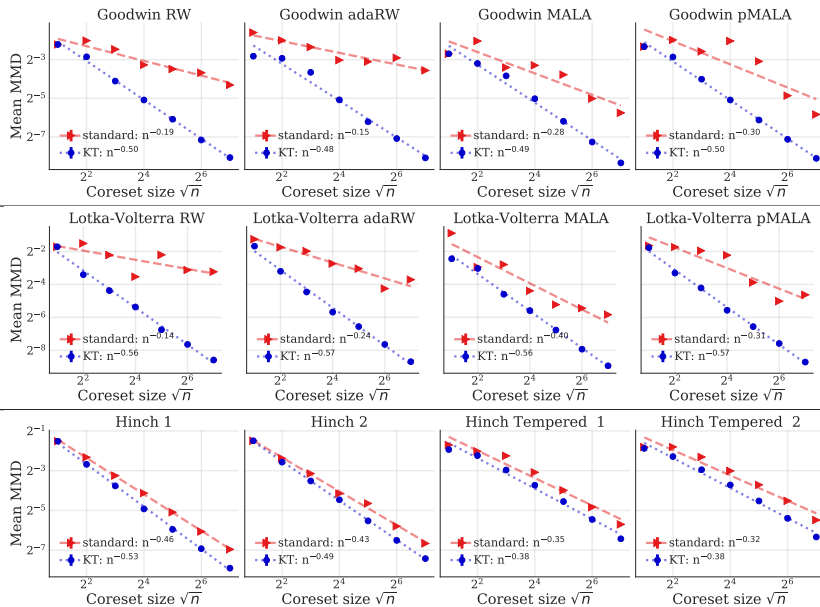
Kernel Thinning vs. Standard MCMC Thinning

Posterior inference for systems of ordinary differential equations (ODEs)

- $\mathbb{P}$ = posterior distribution of coupled ODE model parameters given observed data
- **Goodwin model** of oscillatory enzymatic control ($d = 4$) [Goodwin, 1965]
- **Lotka-Volterra model** of oscillatory predator-prey evolution ($d = 4$) [Lotka, 1925, Volterra, 1926]
- **Hinch model** of cardiac calcium signalling ($d = 38$) [Hinch, Greenstein, Tanskanen, Xu, and Winslow, 2004]
 - Downstream goal: propagate model uncertainty through whole-heart simulation
 - Every sample point discarded via compression = **1000 CPU hours saved**

MCMC input points [Riabiz, Chen, Cockayne, Swietach, Niederer, Mackey, and Oates, 2021]

- **Gaussian random walk** (RW), **adaptive RW** (adaRW) [Haario, Saksman, and Tamminen, 1999]
 - **2 weeks of CPU time** to generate each RW Hinch chain of length 4×10^6
- **Metropolis-adjusted Langevin algorithm** (MALA) [Roberts and Tweedie, 1996]
- **Pre-conditioned MALA** (pMALA) [Girolami and Calderhead, 2011]
- $k(x, y) = \exp(-\frac{1}{2\sigma^2}\|x - y\|_2^2)$ with median heuristic σ^2 [Garreau, Jitkrittum, and Kanagawa, 2017]



KT improves rate of decay and magnitude of MMD, even when standard thinning is accurate

Something's Wrong

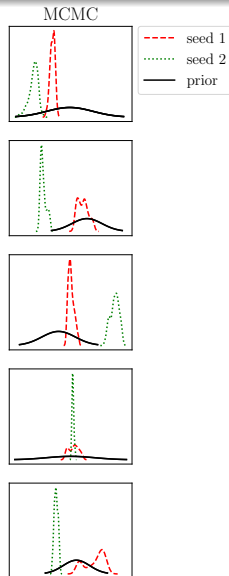
Problem: The Hinch Markov chains haven't mixed!

Solution: Use a more diffuse *tempered* posterior $\tilde{\mathbb{P}}$ for faster mixing

Problem: Tempering introduces a persistent bias

- MCMC points $\mathbb{P}_n$ will be summarizing the wrong distribution $\tilde{\mathbb{P}}$

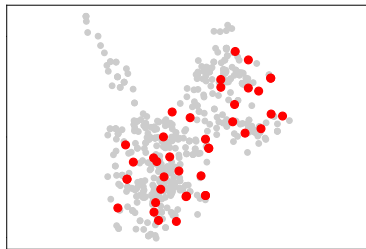
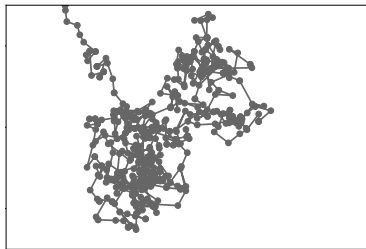
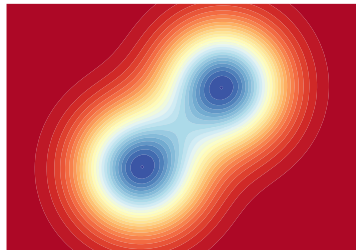
Question: Can we correct for such biases during compression?



Compression with Bias Correction

Question: Can we correct for distributional biases in $\mathbb{P}_n$ during compression?

- e.g., Biases due to off-target sampling, tempering, approximate MCMC, or burn-in



Difficulty: $\mathbb{P}_n$ alone is insufficient; need to measure distance to the true target $\mathbb{P}$

Measuring Distance to $\mathbb{P}$

Quality measure: Maximum mean discrepancy (MMD) [Gretton, Borgwardt, Rasch, Schölkopf, and Smola, 2012]

$$\text{MMD}_{\mathbf{k}}(\mathbb{P}, \mathbb{Q}) = \sup_{\|f\|_{\mathbf{k}} \leq 1} |\mathbb{P}f - \mathbb{Q}f| = \sqrt{(\mathbb{P} \times \mathbb{P})\mathbf{k} + (\mathbb{Q} \times \mathbb{Q})\mathbf{k} - 2(\mathbb{Q} \times \mathbb{P})\mathbf{k}}$$

Problem: Integration under $\mathbb{P}$ is typically intractable!

$\Rightarrow \mathbb{P}\mathbf{k}$ and $\text{MMD}_{\mathbf{k}}(\mathbb{P}, \mathbb{Q})$ cannot be computed in practice for most kernels

Idea: Only consider kernels $\mathbf{k}_{\mathbb{P}}$ with $\mathbb{P}\mathbf{k}_{\mathbb{P}}$ known *a priori* to be 0

- Then MMD computation only depends on $\mathbb{Q}$!

Kernel Stein Discrepancies

Idea: Consider $\text{MMD}_{\mathbf{k}_{\mathbb{P}}}$ with $\mathbb{P}\mathbf{k}_{\mathbb{P}}$ known *a priori* to be 0

Kernel Stein discrepancy (KSD)

[Chwialkowski, Strathmann, and Gretton, 2016, Liu, Lee, and Jordan, 2016, Gorham and Mackey, 2017]

- $\mathbf{k}_{\mathbb{P}}(x, y) = \sum_{j=1}^d \frac{1}{p(x)p(y)} \nabla_{x_j} \nabla_{y_j} (p(x)\mathbf{k}(x, y)p(y))$ [Oates, Girolami, and Chopin, 2017]
 - $\mathbb{P}$ has differentiable Lebesgue density p
 - $\mathbf{k}$ is a bounded base kernel with bounded continuous derivatives
 - $\mathbb{P}\mathbf{k}_{\mathbb{P}} = 0$ whenever $\nabla \log p$ is integrable [Gorham and Mackey, 2017]
 - Depends on $\mathbb{P}$ through $\nabla \log p$: **computable when normalization constant unknown**
- $\Rightarrow$ Kernel Stein discrepancy $\text{MMD}_{\mathbf{k}_{\mathbb{P}}}(\mathbb{P}, \mathbb{Q})$ is **computable**!

Theorem (KSD controls convergence in distribution

[Gorham and Mackey, 2017, Chen, Barp, Briol, Gorham, Girolami, Mackey, and Oates, 2019])

Consider the base kernel $\mathbf{k}(x, y) = (1 + \|x - y\|_2^2)^{-1/2}$. If $\mathbb{P}$ has Lipschitz $\nabla \log p$ and strongly log concave tails, then $\mathbb{Q}_s \Rightarrow \mathbb{P}$ whenever $\text{MMD}_{\mathbf{k}_{\mathbb{P}}}(\mathbb{P}, \mathbb{Q}_s) \rightarrow 0$.

Stein Kernel Thinning: Compression with Bias Correction

(1) Stein thinning: Greedily minimize KSD using points from $\mathcal{S}_{\text{in}} = \{x_1, \dots, x_n\}$

[Riabiz, Chen, Cockayne, Swietach, Niederer, Mackey, and Oates, 2021]

- Choose initial approximation $\mathbb{P}_1 = \delta_{y_1}$ with

$$y_1 \in \operatorname{argmin}_{y \in \mathcal{S}_{\text{in}}} \operatorname{MMD}_{\mathbf{k}_{\mathbb{P}}}(\mathbb{P}, \delta_y) = \operatorname{argmin}_{y \in \mathcal{S}_{\text{in}}} \mathbf{k}_{\mathbb{P}}(y, y)$$

- Iteratively construct **debiased** $\mathbb{P}_n = \frac{1}{n} \sum_{i=1}^n \delta_{y_i}$ with

$$\begin{aligned} y_n &\in \operatorname{argmin}_{y \in \mathcal{S}_{\text{in}}} \operatorname{MMD}_{\mathbf{k}_{\mathbb{P}}}(\mathbb{P}, \frac{n-1}{n} \mathbb{P}_{n-1} + \frac{1}{n} \delta_y) \\ &= \operatorname{argmin}_{y \in \mathcal{S}_{\text{in}}} \mathbf{k}_{\mathbb{P}}(y, y) + 2 \sum_{i=1}^{n-1} \mathbf{k}_{\mathbb{P}}(y_i, y) \end{aligned}$$

- Same point x_i can be selected multiple times
- Runtime = $\mathcal{O}(n^2)$ after n steps

(2) Kernel thinning: Compress debiased approximation $\mathbb{P}_n$ to obtain coreset $\mathbb{Q}$

Stein Kernel Thinning Guarantees

Theorem (Stein KT guarantee [Li, Dwivedi, and Mackey, 2024a])

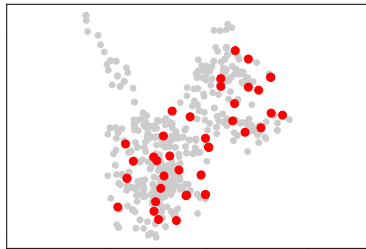
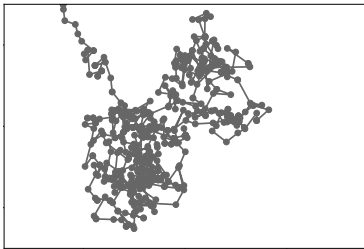
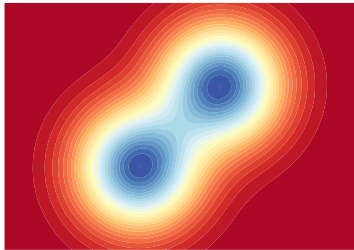
$$\text{MMD}_{\mathbf{k}_{\mathbb{P}}}(\mathbb{P}, \mathbb{Q}) \leq \inf_{w \in \Delta_{n-1}} \text{MMD}_{\mathbf{k}_{\mathbb{P}}}(\mathbb{P}, \sum_{i=1}^n w_i \delta_{x_i}) + \tilde{\mathcal{O}}(s^{-1})$$

with high probability for slow-growing $\mathbf{k}_{\mathbb{P}}$.

Takeaway: Stein KT performs nearly as well as **best simplex reweighting of $\mathcal{S}_{\text{in}}$**

⇒ Nearly as well as Markov chain **with burn-in removed!**

⇒ Nearly as well as off-target sample **after optimal importance sampling reweighting!**



Stein Kernel Thinning Guarantees

Takeaway: Stein KT performs nearly as well as **best simplex reweighting** of $\mathcal{S}_{\text{in}}$

⇒ Nearly as well as Markov chain **with burn-in removed!**

⇒ Nearly as well as off-target sample **after optimal importance sampling reweighting!**

Theorem (Stein KT corrects off-target sampling [Li, Dwivedi, and Mackey, 2024a])

If $\mathcal{S}_{\text{in}}$ drawn i.i.d. from $\tilde{\mathbb{P}}$ with tails no lighter than $\mathbb{P}$ (i.e., $\mathbb{E}_{\mathbb{P}}[\frac{d\mathbb{P}}{d\tilde{\mathbb{P}}}(X)^3 \mathbf{k}_{\mathbb{P}}(X, X)^4] < \infty$),

$$\text{MMD}_{\mathbf{k}_{\mathbb{P}}}(\mathbb{P}, \mathbb{Q}) = \tilde{O}(n^{-1/2}) \text{ in probability,}$$

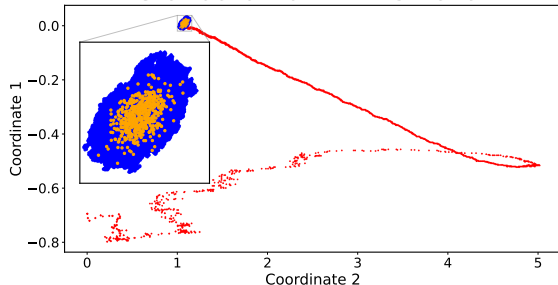
for $s = \sqrt{n}$ and slow-growing $\mathbf{k}_{\mathbb{P}}$.

- Result extends to geometrically ergodic Markov chains targeting $\tilde{\mathbb{P}}$

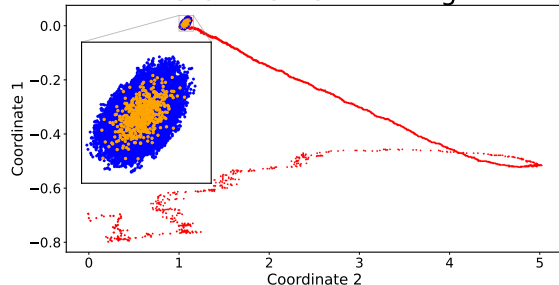
Stein KT in Action: Correcting for Burn-in

Goodwin model of oscillatory enzymatic control

Standard Burn-in Removal



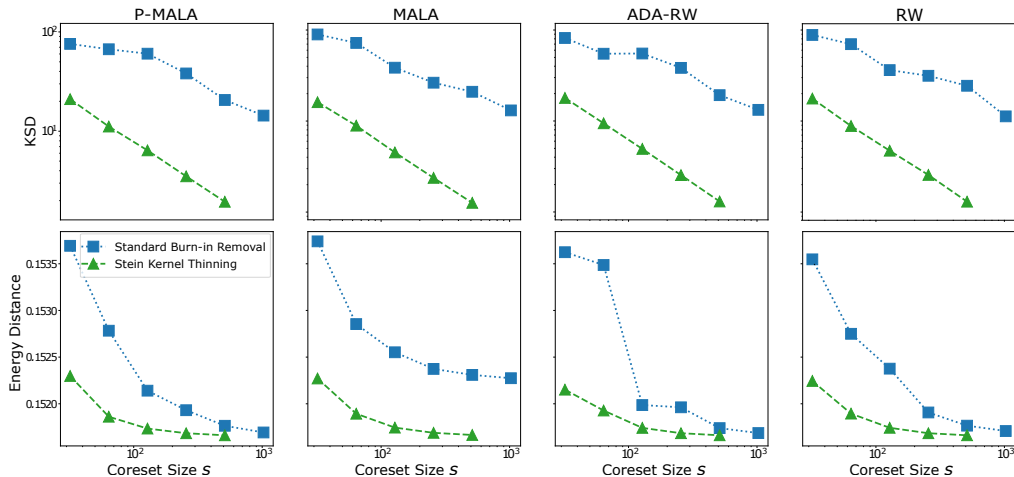
Stein Kernel Thinning



- First two coordinates of P-MALA MCMC output
- Before selecting **coresets**, burn-in removal uses 6 Markov chains to discard **burn-in**
- Stein KT identifies the same **high-density region** with 1 chain

Stein KT in Action: Correcting for Burn-in

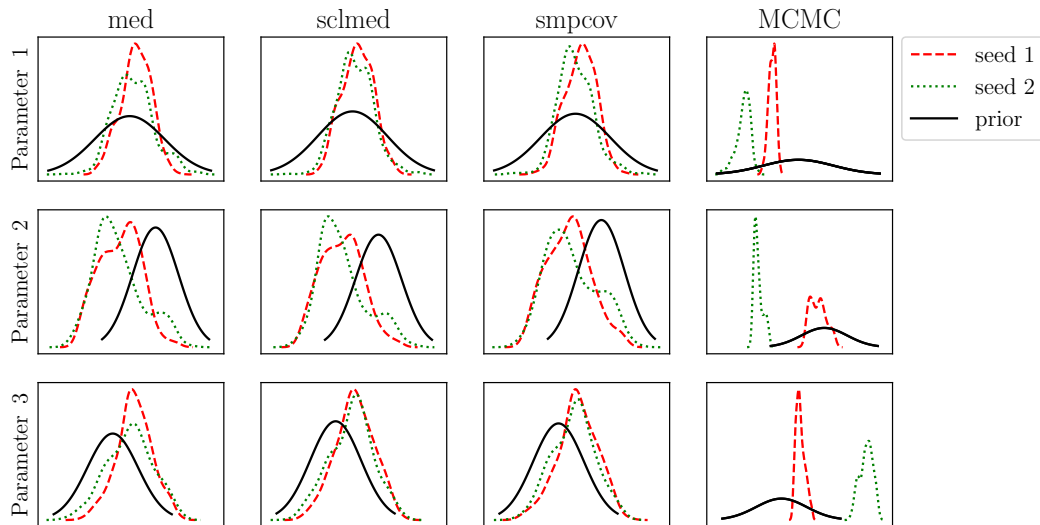
Goodwin model of oscillatory enzymatic control



Stein KT outperforms standard burn-in removal in terms of KSD and energy distance

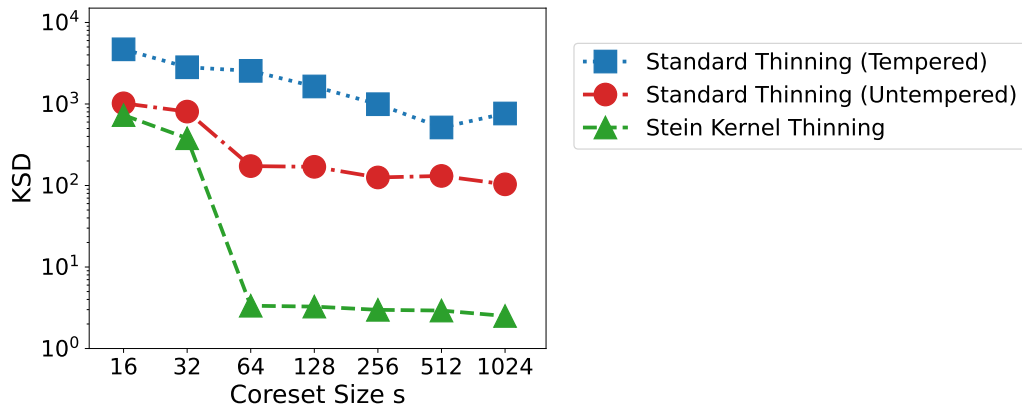
Stein KT in Action: Correcting for Tempering

Hinch model of cardiac calcium signalling: Tempering improves mixing



Stein KT in Action: Correcting for Tempering

Hinch model of cardiac calcium signalling



- Untempered standard thinning yields poor summary due to **poor mixing**
- Tempered thinning without bias correction is even worse (due to **tempering bias**)
- **Tempering + Stein KT bias correction improves approximation to $\mathbb{P}$**

Conclusions

Summary

- New tools for summarizing a probability distribution more effectively than i.i.d. sampling or standard MCMC thinning
- **Kernel thinning** compresses an n point summary into a $\sqrt{n}$ point summary with better-than-i.i.d. approximation error
- **Stein kernel thinning** simultaneously compresses and reduces biases due to off-target sampling, tempering, or burn-in
- **Compress++** speeds up thinning algorithms without ruining their quality

Kernel Thinning and Compress++

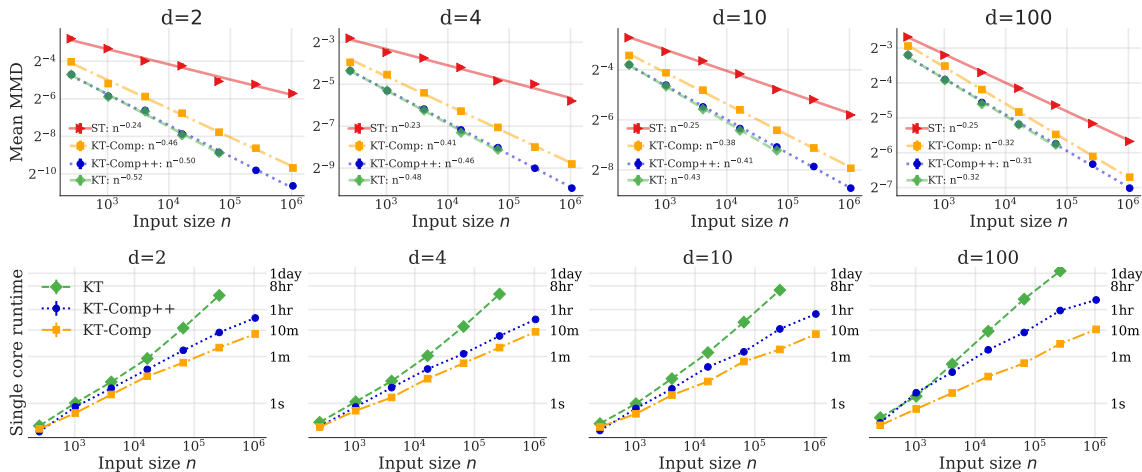
Papers: $\left\{ \begin{array}{l} \text{arxiv.org/abs/2105.05842} \\ \text{arxiv.org/abs/2110.01593} \\ \text{arxiv.org/abs/2111.07941} \end{array} \right.$

Stein Thinning and Stein KT

Papers: $\left\{ \begin{array}{l} \text{arxiv.org/abs/2005.03952} \\ \text{arxiv.org/abs/2404.12290} \end{array} \right.$

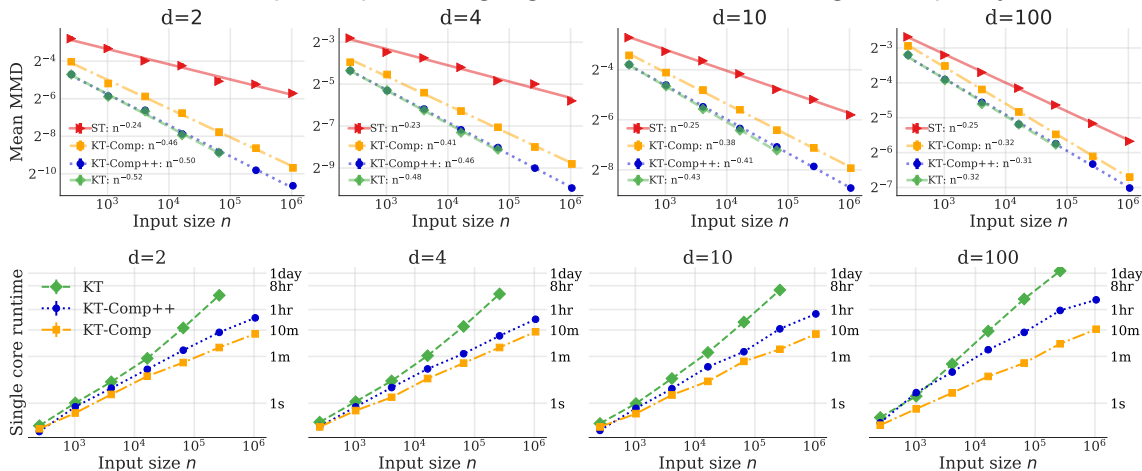
Package: github.com/microsoft/goodpoints

Question: Can we speed up thinning algorithms without ruining their quality?



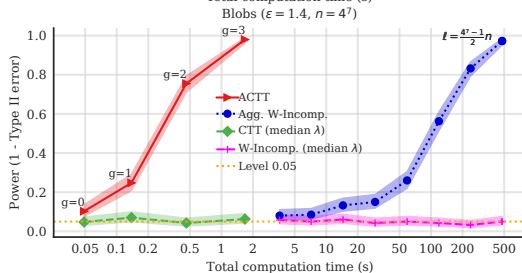
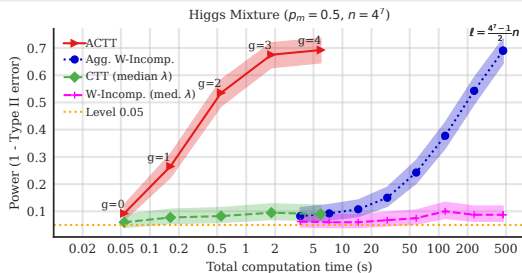
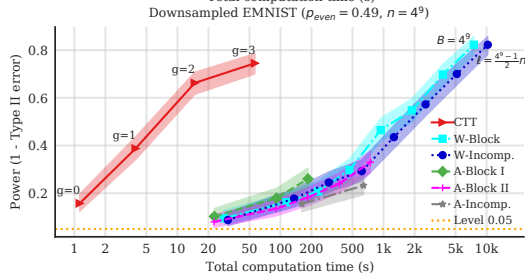
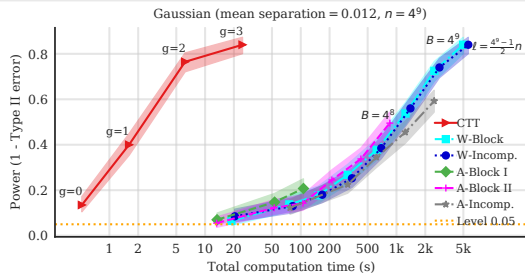
Compress++ orchestrates a sequence of **cheap** halving calls on bins of size 4, 8, 16, ...

Question: Can we speed up thinning algorithms without ruining their quality?



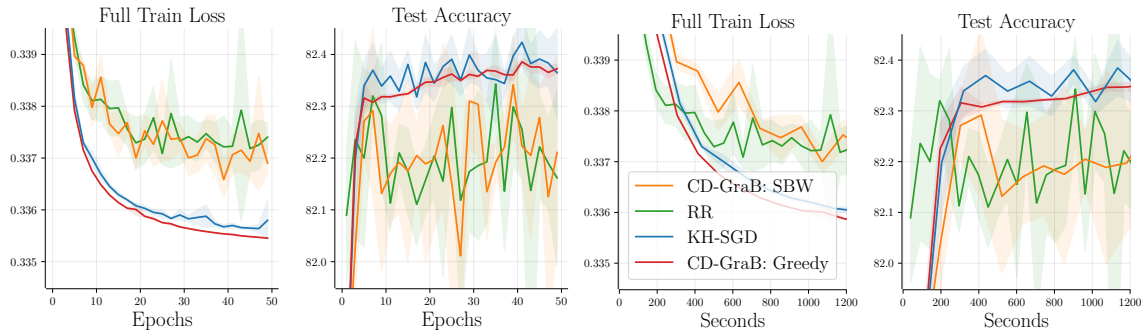
Compress++ reduces n^2 runtime to $n \log^3 n$, applies to any halving algorithm, and inflates error by at most a factor of 4

Compress Then Test [Domingo-Enrich, Dwivedi, and Mackey, 2023]



Compress Then Test (► above) yields powerful kernel tests in near-linear time

Compress Then Sort [Carrell, Gong, Shetty, Dwivedi, and Mackey, 2025]



Inspired by gradient balancing (**CD-GraB**) [Cooper, Guo, Pham, Yuan, Ruan, Lu, and De Sa, 2023], **Kernel Halving SGD** accelerates model training by reordering stochastic gradients

Compress Then Attend [Carrell, Gong, Shetty, Dwivedi, and Mackey, 2025]

Attention Algorithm	Top-1 Accuracy (%)	Layer 1 Runtime (ms)	Layer 2 Runtime (ms)
Exact	82.55 ± 0.00	18.48 ± 0.12	1.40 ± 0.01
Performer	80.56 ± 0.30	2.54 ± 0.01	0.60 ± 0.01
Reformer	81.47 ± 0.06	7.84 ± 0.03	1.53 ± 0.01
KDEformer	82.00 ± 0.07	5.39 ± 0.03	2.28 ± 0.03
Scatterbrain	82.05 ± 0.08	6.86 ± 0.02	1.55 ± 0.03
Thinformer (Ours)	82.18 ± 0.05	2.06 ± 0.01	0.54 ± 0.00

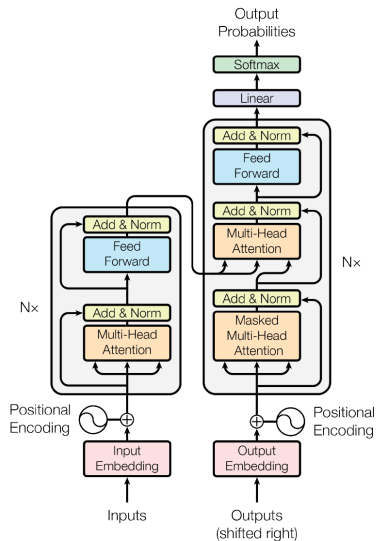
$$\text{Attention} = \text{softmax} \left(\underbrace{\begin{bmatrix} \text{query}_1 \\ \text{query}_2 \\ \vdots \\ \text{query}_n \end{bmatrix} \begin{bmatrix} | & | & & | \\ \text{key}_1 & \text{key}_2 & \cdots & \text{key}_n \\ | & | & & | \end{bmatrix}}_{n \times n} \right) \begin{bmatrix} \text{value}_1 \\ \text{value}_2 \\ \vdots \\ \text{value}_n \end{bmatrix}$$

Thinformer provides a fast, high-quality approximation to attention in transformers

Motivation: Transformers

[Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, and Polosukhin, 2017]

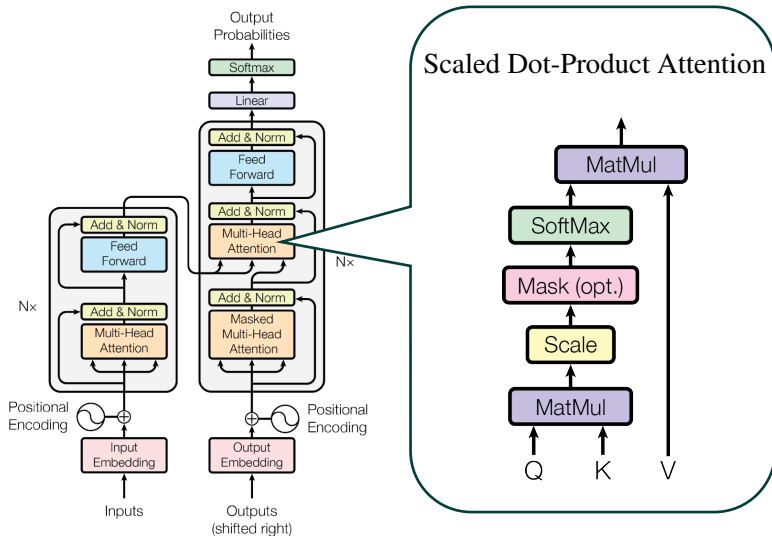
Transformers revolutionized the way we process text, images, and speech



Motivation: Transformers

[Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, and Polosukhin, 2017]

Transformers rely on **attention** to effectively capture dependencies in a sequence



Unpacking Attention

Given: n query, key, and value vectors $(\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i)$ in $\mathbb{R}^d$

$$\text{Attention} = \text{softmax} \underbrace{\begin{pmatrix} \begin{bmatrix} - & \mathbf{q}_1 & - \\ - & \mathbf{q}_2 & - \\ & \vdots & \\ - & \mathbf{q}_n & - \end{bmatrix} \begin{bmatrix} | & | & & | \\ \mathbf{k}_1^\top & \mathbf{k}_2^\top & \dots & \mathbf{k}_n^\top \\ | & | & & | \end{bmatrix} & \begin{bmatrix} - & \mathbf{v}_1 & - \\ - & \mathbf{v}_2 & - \\ & \vdots & \\ - & \mathbf{v}_n & - \end{bmatrix} \end{pmatrix}}_{n \times n}$$

Benefit: Unprecedented success in capturing long-range dependencies

Drawback: Attention is expensive!

① **Time cost:** Grows quadratically in sequence length!

- Long user inputs take too long to process (and consume too much energy)
- Long model generations take too long to produce (and consume too much energy)

Unpacking Attention

Given: n query, key, and value vectors $(\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i)$ in $\mathbb{R}^d$

$$\text{Attention} = \text{softmax} \underbrace{\begin{pmatrix} \begin{bmatrix} - & \mathbf{q}_1 & - \\ - & \mathbf{q}_2 & - \\ & \vdots & \\ - & \mathbf{q}_n & - \end{bmatrix} & \begin{bmatrix} | & | & & | \\ \mathbf{k}_1^\top & \mathbf{k}_2^\top & \dots & \mathbf{k}_n^\top \\ | & | & & | \end{bmatrix} \end{pmatrix}}_{n \times n} \begin{bmatrix} - & \mathbf{v}_1 & - \\ - & \mathbf{v}_2 & - \\ & \vdots & \\ - & \mathbf{v}_n & - \end{bmatrix}$$

Benefit: Unprecedented success in capturing long-range dependencies

Drawback: Attention is expensive!

② **Memory cost:** Grows linearly with sequence length!

- Limits both the maximum context length and the longest feasible generation

Unpacking Attention

Given: n query, key, and value vectors $(\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i)$ in $\mathbb{R}^d$

$$\text{Attention} = \text{softmax} \underbrace{\begin{pmatrix} \begin{bmatrix} - & \mathbf{q}_1 & - \\ - & \mathbf{q}_2 & - \\ & \vdots & \\ - & \mathbf{q}_n & - \end{bmatrix} & \begin{bmatrix} | & | & & | \\ \mathbf{k}_1^\top & \mathbf{k}_2^\top & \cdots & \mathbf{k}_n^\top \\ | & | & & | \end{bmatrix} \end{pmatrix}}_{n \times n} \begin{bmatrix} - & \mathbf{v}_1 & - \\ - & \mathbf{v}_2 & - \\ & \vdots & \\ - & \mathbf{v}_n & - \end{bmatrix}$$

Benefit: Unprecedented success in capturing long-range dependencies

Drawback: Attention is expensive!

- 1 **Time cost:** Grows quadratically in sequence length!
- 2 **Memory cost:** Grows linearly with sequence length!

Question: Can we approximate attention both cheaply and accurately?

Attention Is Just Averaging

Given: n query, key, and value vectors $(\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i)$ in $\mathbb{R}^d$

$$\mathbf{Attention}_{ij} = \frac{\frac{1}{n} \sum_{l=1}^n \exp(\mathbf{q}_i \cdot \mathbf{k}_l) \mathbf{v}_{lj}}{\frac{1}{n} \sum_{l=1}^n \exp(\mathbf{q}_i \cdot \mathbf{k}_l)} \approx \frac{\frac{1}{|S|} \sum_{l \in S} \exp(\mathbf{q}_i \cdot \mathbf{k}_l) \mathbf{v}_{lj}}{\frac{1}{|S|} \sum_{l \in S} \exp(\mathbf{q}_i \cdot \mathbf{k}_l)}$$

Idea: Approximate averages using a subset S of $s \ll n$ key-value pairs

Benefits: Only $O(ns)$ work instead of $\Theta(n^2)$ and only s cache entries instead of n

Uniform subset: Attention error $\propto \frac{1}{\sqrt{s}}$; need $s = 10000$ for 1% error

Question: Can we do better?

Thinformer = Compressed Kernel Halving

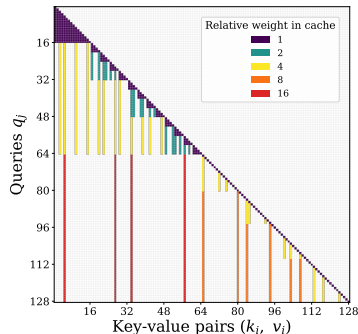
Thinformer [Carrell, Gong, Shetty, Dwivedi, and Mackey, 2025]

- **Input:** Attention kernel $\kappa_{\text{att}}((\mathbf{k}_1, \mathbf{v}_1), (\mathbf{k}_2, \mathbf{v}_2)) = \exp(\mathbf{k}_1 \cdot \mathbf{k}_2) \mathbf{v}_1 \cdot \mathbf{v}_2$
- **Output:** s tokens with subset averages nearly matching full averages in $\tilde{O}(s^2)$ time

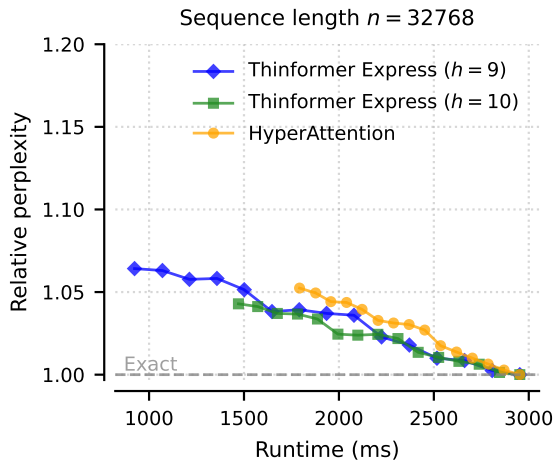
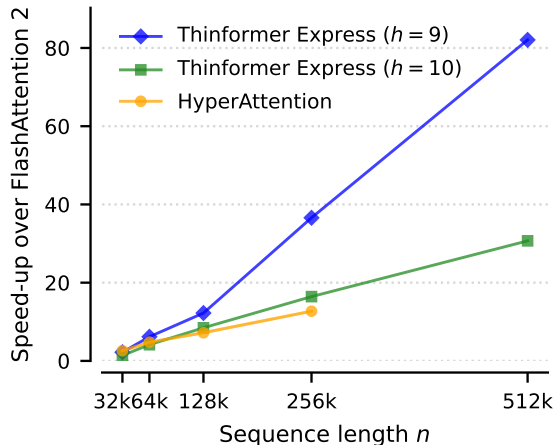
$$\Rightarrow \left| \frac{1}{n} \sum_{l=1}^n \exp(\mathbf{q}_i \cdot \mathbf{k}_l) \mathbf{v}_{lj} - \frac{1}{|S|} \sum_{l \in S} \exp(\mathbf{q}_i \cdot \mathbf{k}_l) \mathbf{v}_{lj} \right| = O\left(\frac{\log(s) \sqrt{\log \log(n)}}{s}\right)$$

Thinformer Express [Gong, Carrell, Dwivedi, and Mackey, 2026]

- Supports **causal masking** and **online generation**
- Maintains coreset of size s in $O(s^2 \log^2(n))$ time
- Guarantees $\tilde{O}(\frac{1}{s})$ masked attention error

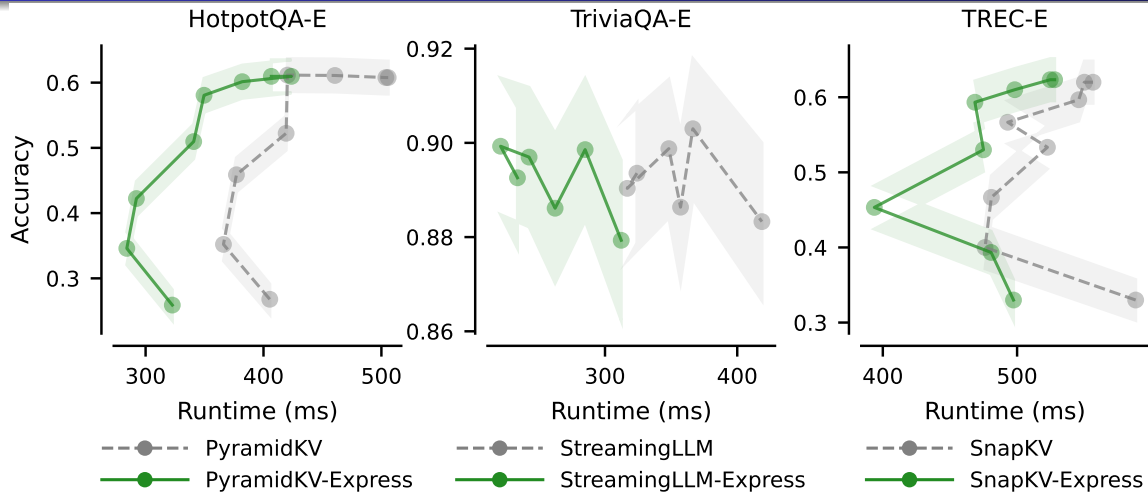


Accelerating Prefill with ChatGLM2-6B-32K ($d = 128$)



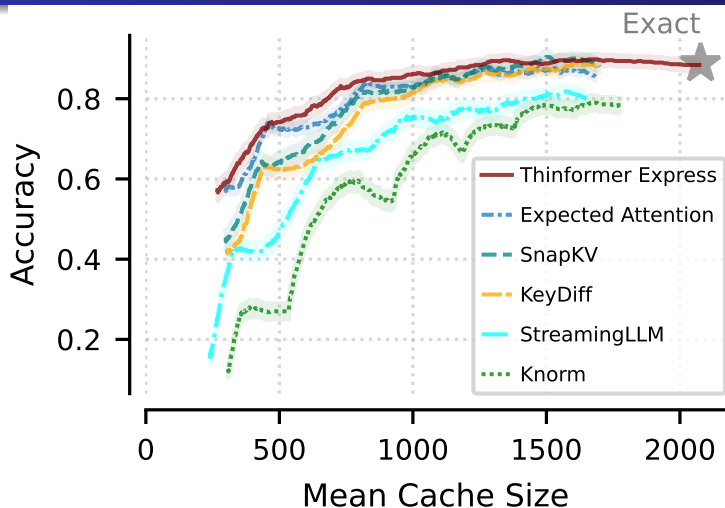
- **HyperAttention:** Heavy hitters + subsampling [Han, Jayaram, Karbasi, Mirrokni, Woodruff, and Zandieh, 2024]

Accelerated KV Cache Compression with Llama 3.1 8B Instruct



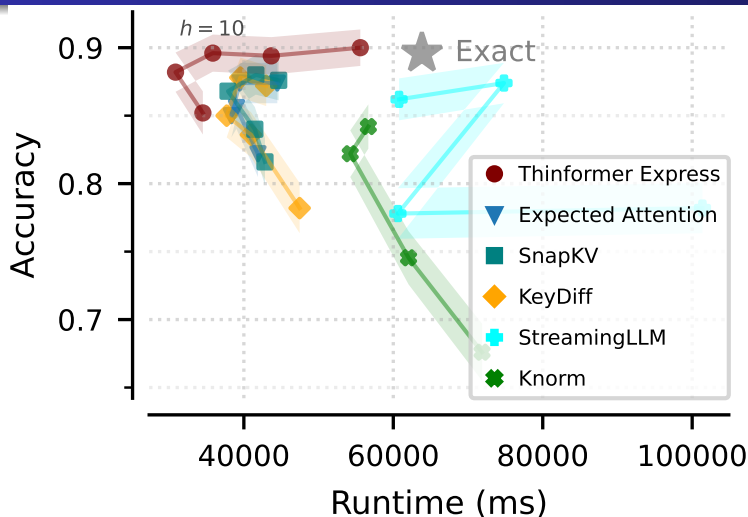
Thinformer Express can **accelerate** leading KV cache compression methods while **preserving** their long-context language understanding quality

Budgeted Reasoning with DeepSeek-R1-Distill-Llama-8B



- **MATH-500:** Competition-level math problems that benefit from step-by-step reasoning
Thinformer Express matches Exact accuracy with **61%** of the cache size

Accelerated Reasoning with DeepSeek-R1-Distill-Llama-8B



- **MATH-500:** Competition-level math problems that benefit from step-by-step reasoning
Thinformer Express matches Exact accuracy with **56%** of the runtime

Weighted vs. Unweighted Approximations

Thinformer outputs s tokens with unweighted averages closely matching exact averages

$$\left| \frac{1}{n} \sum_{l=1}^n \exp(\mathbf{q}_i \cdot \mathbf{k}_l) \mathbf{v}_{lj} - \frac{1}{s} \sum_{l \in S} \exp(\mathbf{q}_i \cdot \mathbf{k}_l) \mathbf{v}_{lj} \right| = \tilde{O}\left(\frac{1}{s}\right)$$

Question: Can we do even better?

Lower bound: $\Omega(\frac{1}{s})$ error is **unavoidable** for unweighted attention approximations

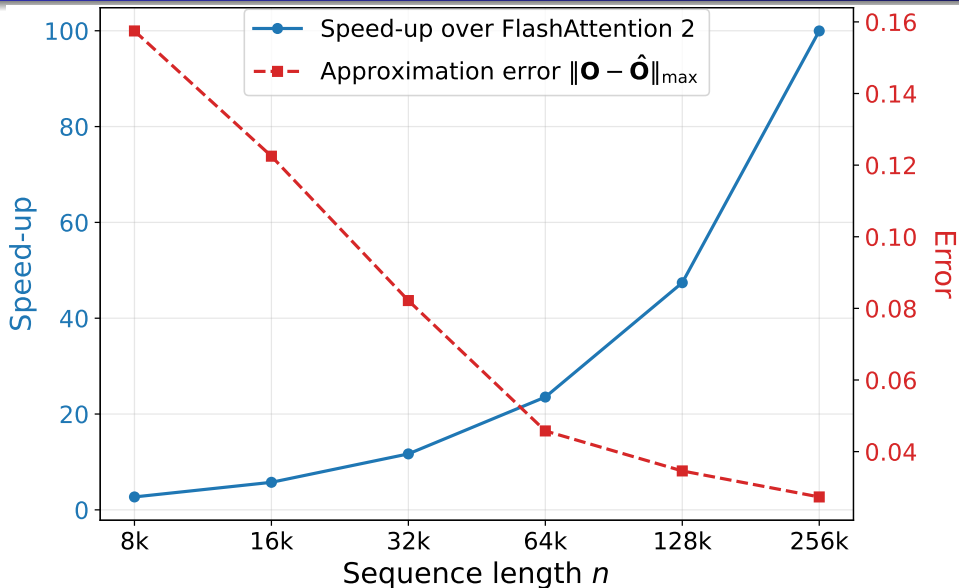
[Liberty, Andoni, and Kleiner, 2026]

Question: Can we do even better...with weights?

WildCat [Schroder and Mackey, 2026]

- Selects **optimally-weighted** keys via divide-and-conquer randomly pivoted Nyström
[Chen, Epperly, Tropp, and Webber, 2025]
- Enables **superpolynomial** $(1/n^{\sqrt{\log \log n}})$ error decay in **near-linear** time

WildCat Speed and Error Both Improve as n Grows ($s = 64$)



KV Cache Compression with Qwen2.5-7B-Instruct

75.0% Compression

Method	qasper	multifield	hotpot	2wiki	gov	multinews	trec	trivia	samsum	p.count	p.ret	lcc	repo-p	average
Exact	43.76	50.08	56.43	43.71	34.16	24.32	64.00	87.53	38.56	14.67	99.67	69.96	62.17	53.00
StreamingLLM	23.17	25.49	26.45	20.89	29.65	22.25	52.33	75.17	35.79	12.50	24.83	68.48	56.22	36.40
PyramidKV	21.59	29.96	39.38	30.24	27.12	21.13	43.33	86.77	38.27	15.88	61.06	67.40	59.20	41.64
BalanceKV	29.50	36.57	37.89	23.71	30.27	21.98	55.00	73.83	34.56	12.67	71.67	65.48	62.57	42.75
Uniform	26.91	37.51	38.46	25.93	30.02	21.86	54.33	81.71	35.37	15.33	63.44	64.84	61.34	42.85
SnapKV	25.52	30.13	44.36	31.80	29.70	22.10	49.33	88.32	37.15	16.67	89.06	69.16	56.33	45.36
WildCat	33.23	38.13	43.43	33.37	30.30	22.26	54.33	86.15	35.38	14.33	98.00	64.85	60.43	47.25

- **13 LongBench-E tasks:** question answering, summarization, code completion, passage retrieval, passage count, and few-shot learning [Bai, Lv, Zhang, Lyu, Tang, Huang, Du, Liu, Zeng, Hou, et al., 2024]
- **StreamingLLM:** Preserves latest tokens and attention sinks [Xiao, Tian, Chen, Han, and Lewis, 2024]
- **PyramidKV:** Varies cache size across layers [Li, Huang, Yang, Venkitesh, Locatelli, Ye, Cai, Lewis, and Chen, 2024b]
- **BalanceKV:** Subsamples tokens using self-balancing walks [Kochetkova, Sheth, Han, Zandieh, and Kapralov, 2026]
- **Uniform:** Subsamples tokens uniformly
- **SnapKV:** Preserves tokens similar to recent queries [Li, Huang, Yang, Venkitesh, Locatelli, Ye, Cai, Lewis, and Chen, 2024b]

Summary

- New tools for summarizing a probability distribution more effectively than i.i.d. sampling or standard MCMC thinning
- [Kernel thinning](#) compresses an n point summary into a $\sqrt{n}$ point summary with better-than-i.i.d. approximation error
- [Stein kernel thinning](#) simultaneously compresses and reduces biases due to off-target sampling, tempering, or burn-in
- [Compress++](#) speeds up thinning algorithms without ruining their quality
- [CTT](#), [KH-SGD](#), [Thinformer](#), & [WildCat](#) accelerate testing, training, & transformers

Code: $\left\{ \begin{array}{ll} \text{github.com/microsoft/goodpoints} & \text{github.com/microsoft/thinformer} \\ \text{github.com/microsoft/khsgd} & \text{github.com/microsoft/wildcat} \end{array} \right.$

Many opportunities for future development

① Value of swapping

- KT-SWAP refinement stage typically leads to significant quality improvements over KT-SPLIT alone. Can we establish stronger guarantees for KT-SWAP?

② Faster debiasing

- Low-rank SKT [Li, Dwivedi, and Mackey, 2024a] matches Stein KT guarantees in $\mathcal{O}(n^{1.5})$ time. Can we improve runtime further?

③ Weighted compression

- For applications that support weights, can we establish stronger guarantees for weighted coresets?
- e.g., weighted Stein Recombination and Stein Cholesky coresets can match SKT guarantees with as few as $s = \text{polylog}(n)$ points instead of $s = \sqrt{n}$ [Li, Dwivedi, and Mackey, 2024a].

④ Other metrics

- For which other metrics is (significantly) better-than-i.i.d. compression achievable?

Related Work on MMD Coresets

Uniform distribution $\mathbb{P}$ on $[0, 1]^d$: L^2 discrepancy MMD, s points

- **Quasi-Monte Carlo** [Chen and Skrganov, 2002]: $\mathcal{O}(s^{-1} \log^{\frac{d-1}{2}} s)$
- **Online Haar strategy** [Dwivedi, Feldheim, Gurel-Gurevich, and Ramdas, 2019]: $\mathcal{O}(s^{-1} \log^{2d} s)$

Order $s^{-\frac{1}{2}}$ MMD coresets for general $\mathbb{P}$

- **i.i.d.** [Tolstikhin, Sriperumbudur, and Muandet, 2017], **geometrically ergodic MCMC** [Dwivedi and Mackey, 2024]
- **Kernel herding** [Chen, Welling, and Smola, 2010, Lacoste-Julien, Lindsten, and Bach, 2015], **Stein points MCMC** [Chen, Barp, Briol, Gorham, Girolami, Mackey, and Oates, 2019], **Greedy sign selection** [Karnin and Liberty, 2019]

Finite-dimensional linear kernels on $\mathbb{R}^d$: $\mathcal{O}(\sqrt{d}s^{-1} \log^{2.5} s)$, s points

- **Discrepancy construction** [Harvey and Samadi, 2014]: does not cover infinite-dimensional $\mathbf{k}$

Unknown coreset quality

- **Super-sampling with a reservoir** [Paige, Sejdinovic, and Wood, 2016]: coreset quality not analyzed
- **Support points** [Mak and Joseph, 2018]
 - Optimal s coreset has $o(s^{-\frac{1}{2}})$ energy distance MMD but no construction given
 - Practical convex-concave procedures not analyzed or shown to be optimal

References I

- R. Alweiss, Y. P. Liu, and M. Sawhney. Discrepancy minimization via a self-balancing walk. *arXiv preprint arXiv:2006.14009*, 2020.
- Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, 2024.
- F. O. Campos, Y. Shiferaw, A. J. Prassl, P. M. Boyle, E. J. Vigmond, and G. Plank. Stochastic spontaneous calcium release events trigger premature ventricular complexes by overcoming electrotonic load. *Cardiovascular Research*, 107(1):175–183, 2015.
- A. M. Carrell, A. Gong, A. Shetty, R. Dwivedi, and L. Mackey. Low-rank thinning. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=iAkg2nVmvN>.
- W. W. L. Chen and M. M. Skrikanov. Explicit constructions in the classical mean squares problem in irregularities of point distribution. *Journal fur die Reine und Angewandte Mathematik*, 545:67–96, 2002.
- W. Y. Chen, A. Barp, F.-X. Briol, J. Gorham, M. Girolami, L. Mackey, and C. Oates. Stein point Markov chain Monte Carlo. In *International Conference on Machine Learning*, pages 1011–1021. PMLR, 2019.
- Y. Chen, M. Welling, and A. Smola. Super-samples from kernel herding. In *Proceedings of the Twenty-Sixth Conference on Uncertainty in Artificial Intelligence*, UAI'10, page 109–116, Arlington, Virginia, USA, 2010. AUAI Press. ISBN 9780974903965.
- Y. Chen, E. N. Epperly, J. A. Tropp, and R. J. Webber. Randomly pivoted cholesky: Practical approximation of a kernel matrix with few entry evaluations. *Communications on Pure and Applied Mathematics*, 78(5):995–1041, 2025.
- K. Chwialkowski, H. Strathmann, and A. Gretton. A kernel test of goodness of fit. In *Proceedings of the 33rd International Conference on Machine Learning*, 2016.
- M. A. Colman. Arrhythmia mechanisms and spontaneous calcium release: Bi-directional coupling between re-entrant and focal excitation. *PLoS Computational Biology*, 15(8), 2019.
- A. F. Cooper, W. Guo, K. Pham, T. Yuan, C. F. Ruan, Y. Lu, and C. De Sa. Coordinating distributed example orders for provably accelerated training. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- C. Domingo-Enrich, R. Dwivedi, and L. Mackey. Compress then test: Powerful kernel testing in near-linear time. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, Proceedings of Machine Learning Research. PMLR, 25–27 Apr 2023.
- R. Dwivedi and L. Mackey. Generalized kernel thinning. In *International Conference on Learning Representations*, 2022.

References II

- R. Dwivedi and L. Mackey. Kernel thinning. *Journal of Machine Learning Research*, 25(152):1–77, 2024.
- R. Dwivedi, O. N. Feldheim, O. Gurel-Gurevich, and A. Ramdas. The power of online thinning in reducing discrepancy. *Probability Theory and Related Fields*, 174(1):103–131, 2019.
- D. Garreau, W. Jitkrittum, and M. Kanagawa. Large sample analysis of the median heuristic. *arXiv preprint arXiv:1707.07269*, 2017.
- M. Girolami and B. Calderhead. Riemann manifold Langevin and Hamiltonian Monte Carlo methods. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 73(2):123–214, 2011.
- A. Gong, A. M. Carrell, R. Dwivedi, and L. Mackey. Express language modeling. *arXiv preprint arXiv:2606.10944*, 2026.
- B. C. Goodwin. Oscillatory behavior in enzymatic control process. *Advances in Enzyme Regulation*, 3:318–356, 1965.
- J. Gorham and L. Mackey. Measuring sample quality with kernels. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773, 2012.
- H. Haario, E. Saksman, and J. Tamminen. Adaptive proposal distribution for random walk Metropolis algorithm. *Computational Statistics*, 14(3):375–395, 1999.
- I. Han, R. Jayaram, A. Karbasi, V. Mirrokni, D. Woodruff, and A. Zandieh. Hyperattention: Long-context attention in near-linear time. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Eh00d2BJIM>.
- N. Harvey and S. Samadi. Near-Optimal Herding. In *Proceedings of The 27th Conference on Learning Theory*, volume 35, pages 1165–1182, 2014.
- R. Hinch, J. Greenstein, A. Tanskanen, L. Xu, and R. Winslow. A simplified local control model of calcium-induced calcium release in cardiac ventricular myocytes. *Biophysical journal*, 87(6):3723–3736, 2004.
- S. Joshi, R. V. Kommaraji, J. M. Phillips, and S. Venkatasubramanian. Comparing distributions and shapes using the kernel distance. In *Proceedings of the twenty-seventh annual symposium on Computational geometry*, pages 47–56, 2011.
- Z. Karnin and E. Liberty. Discrepancy, coresets, and sketches in machine learning. In *Conference on Learning Theory*, pages 1975–1993. PMLR, 2019.
- E. Kochetkova, K. J. Sheth, I. Han, A. Zandieh, and M. Kapralov. Streaming attention approximation via discrepancy theory. *Advances in Neural Information Processing Systems*, 38:20935–20972, 2026.
- S. Lacoste-Julien, F. Lindsten, and F. Bach. Sequential kernel herding: Frank-Wolfe optimization for particle filtering. In *Artificial Intelligence and Statistics*, pages 544–552. PMLR, 2015.

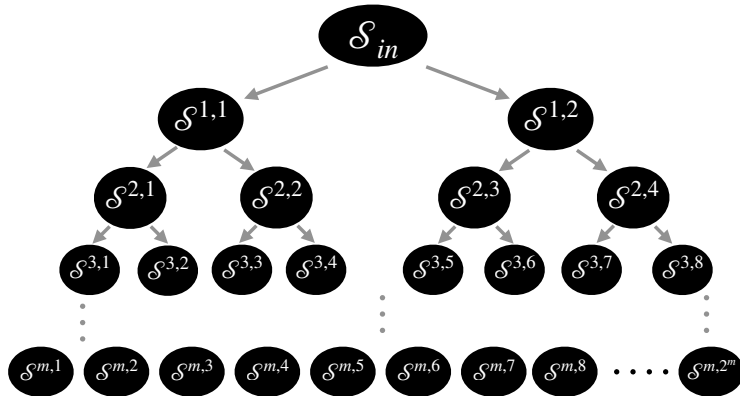
References III

- L. Li, R. Dwivedi, and L. Mackey. Debiased distribution compression. In *Proceedings of the 41st International Conference on Machine Learning*, volume 203 of *Proceedings of Machine Learning Research*. PMLR, 21–27 Jul 2024a.
- Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024b.
- E. Liberty, A. Andoni, and E. Kleiner. Nearly optimal attention coresets. *arXiv preprint arXiv:2605.05602*, 2026.
- Q. Liu, J. D. Lee, and M. I. Jordan. A kernelized Stein discrepancy for goodness-of-fit tests and model evaluation. In *Proceedings of the 33rd International Conference on Machine Learning*, 2016.
- A. J. Lotka. *Elements of physical biology*. Williams & Wilkins, 1925.
- S. Mak and V. R. Joseph. Support points. *The Annals of Statistics*, 46(6A):2562–2592, 2018.
- S. A. Niederer, J. Lumens, and N. A. Trayanova. Computational models in cardiology. *Nature Reviews Cardiology*, 16(2):100–111, 2019.
- S. A. Niederer, M. S. Sacks, M. Girolami, and K. Willcox. Scaling digital twins from the artisanal to the industrial. *Nature Computational Science*, 1(5):313–320, 2021.
- C. J. Oates, M. Girolami, and N. Chopin. Control functionals for Monte Carlo integration. *Journal of the Royal Statistical Society, Series B*, 79(3):695–718, 2017.
- B. Paige, D. Sejdinovic, and F. Wood. Super-sampling with a reservoir. In *Proceedings of the Thirty-Second Conference on Uncertainty in Artificial Intelligence*, pages 567–576, 2016.
- J. M. Phillips. ϵ -samples for kernels. In *Proceedings of the twenty-fourth annual ACM-SIAM symposium on Discrete algorithms*, pages 1622–1632. SIAM, 2013.
- J. M. Phillips and W. M. Tai. Improved coresets for kernel density estimates. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 2718–2727. SIAM, 2018.
- J. M. Phillips and W. M. Tai. Near-optimal coresets of kernel density estimates. *Discrete & Computational Geometry*, 63(4):867–887, 2020.
- M. Riabiz, W. Chen, J. Cockayne, P. Swietach, S. A. Niederer, L. Mackey, and C. Oates. Optimal thinning of MCMC output. *To appear: Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 2021.
- G. O. Roberts and R. L. Tweedie. Exponential convergence of Langevin distributions and their discrete approximations. *Bernoulli*, 2(4):341–363, 1996.

References IV

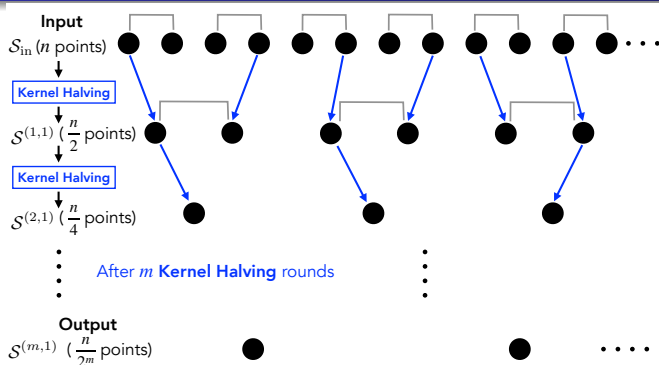
- T. Schroder and L. Mackey. Wildcat: Near-linear attention in theory and practice. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=lfqyLp4hZm>.
- A. Shetty, R. Dwivedi, and L. Mackey. Distribution compression in near-linear time. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/pdf?id=lzupY5zjaU9>.
- W. M. Tai. New nearly-optimal coresets for kernel density estimation. *arXiv preprint arXiv:2007.08031*, 2020.
- I. Tolstikhin, B. K. Sriperumbudur, and K. Muandet. Minimax estimation of kernel mean embeddings. *The Journal of Machine Learning Research*, 18(1): 3002–3048, 2017.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.
- V. Volterra. Variazioni e fluttuazioni del numero d'individui in specie animali conviventi. 1926.
- G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sSS1zVF>.

KT-SPLIT



KT-SPLIT partitions the input $\mathcal{S}_{\text{in}}$ recursively, first dividing the input sequence in half, then halving those halves into quarters, and so on

- **Runs online:** after i input points processed have output coresets of size $\frac{i}{2^m}$



Each output coreset $S^{(m,\ell)}$ is the result of repeated **kernel halving**

- On each halving round, remaining points are paired, and one point from each pair is selected using a new Hilbert space generalization of the self-balancing walk of Alweiss, Liu, and Sawhney [2020]
- Selection rule ensures that $\mathbb{P}_n \mathbf{k} - \mathbb{Q} \mathbf{k}$ remains small with high probability

Kernel Halving with a Self-Balancing Hilbert Walk

Algorithm: Self-balancing Hilbert Walk [Dwivedi and Mackey, 2024]

Input: sequence of functions $(f_i)_{i=1}^{n/2}$ in Hilbert space $\mathcal{H}$, threshold sequence $(\mathfrak{a}_i)_{i=1}^{n/2}$

$\psi_0 \leftarrow \mathbf{0} \in \mathcal{H}$

for $i = 1, 2, \dots, n/2$ **do**

$\alpha_i \leftarrow \langle \psi_{i-1}, f_i \rangle_{\mathcal{H}}$ // Compute Hilbert space inner product

if $|\alpha_i| > \mathfrak{a}_i$:

$\psi_i \leftarrow \psi_{i-1} - f_i \cdot \alpha_i / \mathfrak{a}_i$ // We choose $\mathfrak{a}_i$ to avoid this case with high probability

else:

$\eta_i \leftarrow 1$ with probability $\frac{1}{2}(1 - \alpha_i / \mathfrak{a}_i)$ and $\eta_i \leftarrow -1$ otherwise

$\psi_i \leftarrow \psi_{i-1} + \eta_i f_i$

end

return $\psi_{n/2}$, sum of signed input functions // $\psi_{n/2} = \sum_{i=1}^{n/2} \eta_i f_i$ with high probability

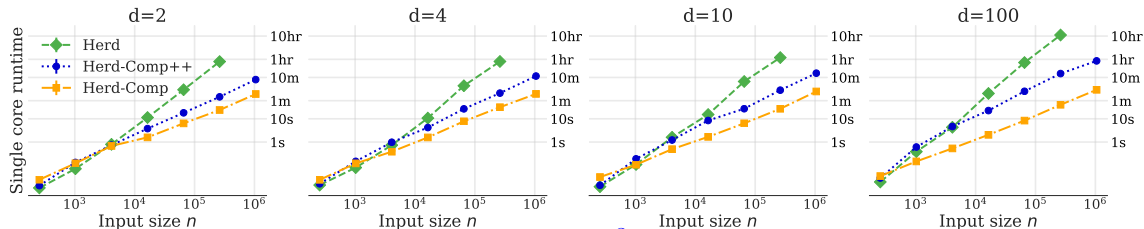
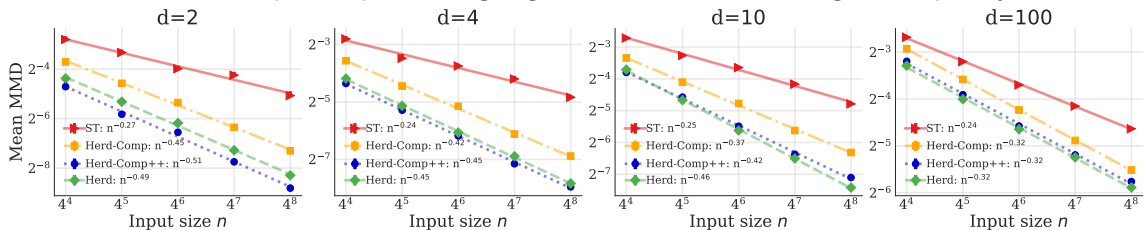
① **Kernel Halving:** If $f_i = \mathbf{k}(x_{2i-1}, \cdot) - \mathbf{k}(x_{2i}, \cdot)$, half of input points $\mathcal{S}_{\text{out}}$ given sign 1

$$\Rightarrow \frac{1}{n} \psi_{n/2} = \mathbb{P}_n \mathbf{k} - \mathbb{Q} \mathbf{k} \text{ with } \mathbb{Q} = \frac{2}{n} \sum_{x \in \mathcal{S}_{\text{out}}} \delta_x$$

② **Balance:** If $\mathcal{H} = \mathbf{k}$ RKHS, $\langle \mathbb{P}_n \mathbf{k} - \mathbb{Q} \mathbf{k}, f \rangle_{\mathcal{H}}$ is $\mathcal{O}(\sqrt{\log(n)}/n)$ sub-Gaussian, $\forall f \in \mathcal{H}$

• In contrast, i.i.d. signs η_i give $\mathbb{P}_n f - \mathbb{Q} f = \langle \mathbb{P}_n \mathbf{k} - \mathbb{Q} \mathbf{k}, f \rangle_{\mathcal{H}} = \Omega(1/\sqrt{n})$

Question: Can we speed up thinning algorithms without ruining their quality?



Compress++ reduces n^2 runtime to $n \log^3 n$, applies to any thinning algorithm (e.g., **kernel herding**), and inflates error by at most a factor of 4

Algorithm 2: COMPRESS: Given n points return thinned coreset of size $\sqrt{n}$

Input: halving algorithm HALVE, point sequence $\mathcal{S}_{\text{in}}$ of size n

if $n = 1$ **then return** $\mathcal{S}_{\text{in}}$

Partition $\mathcal{S}_{\text{in}}$ into four arbitrary subsequences $\{\mathcal{S}_i\}_{i=1}^4$ each of size $n/4$

for $i = 1, 2, 3, 4$ **do**

$\tilde{\mathcal{S}}_i \leftarrow \text{COMPRESS}(\mathcal{S}_i, \text{HALVE})$ // return coresets of size $\sqrt{\frac{n}{4}}$

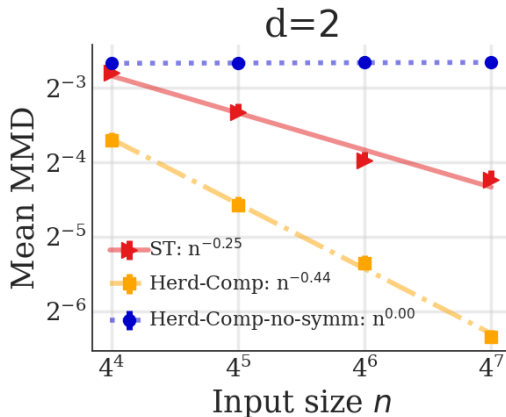
end

$\tilde{\mathcal{S}} \leftarrow \text{CONCATENATE}(\tilde{\mathcal{S}}_1, \tilde{\mathcal{S}}_2, \tilde{\mathcal{S}}_3, \tilde{\mathcal{S}}_4)$ // coreset of size $2\sqrt{n}$

return $\text{HALVE}(\tilde{\mathcal{S}})$ // coreset of size $\sqrt{n}$

Error guarantees rely on unbiased halving ($\mathbb{E}[\mathbb{P}_{\text{Halve}} \mathbf{k} \mid \mathcal{S}_{\text{in}}] = \mathbb{P}_{\text{in}} \mathbf{k}$)

- Achieved for any halving algorithm by [symmetrization](#): return either the outputted half or its complement with equal probability



Related Work on L^∞ Coresets

L^∞ coresets for $\mathbb{P}_n$: $o(n^{-\frac{1}{4}})$ L^∞ error, $\sqrt{n}$ points

- Series of breakthroughs due to [Joshi, Kommaraji, Phillips, and Venkatasubramanian, 2011, Phillips, 2013, Phillips and Tai, 2018, 2020, Tai, 2020]

Best known L^∞ guarantees (for coreset of size $\sqrt{n}$)

- Phillips and Tai [2020]: $\mathcal{O}(\sqrt{d}n^{-\frac{1}{2}}\sqrt{\log n})$ error, $\Omega(n^4)$ time, $\Omega(n^2)$ space
- Tai [2020] (Gaussian $\mathbf{k}$): $\mathcal{O}(2^d n^{-\frac{1}{2}}\sqrt{\log(d \log n)})$ error, $\Omega(\max(d^{5d}, n^4))$ time
- Both are offline and require rebalancing after approximate halving steps
- This work: $\mathcal{O}(\sqrt{d}n^{-\frac{1}{2}}\log n)$ error, $\mathcal{O}(n^2)$ time, $\mathcal{O}(nd)$ space, online, exact halving
 - Sub-Gaussian $(\mathbf{k}_{\text{rt}}, \mathbb{P})$: $\mathcal{O}(\sqrt{d}n^{-\frac{1}{2}}\sqrt{\log n \log \log n})$ error
 - Compact support $(\mathbf{k}_{\text{rt}}, \mathbb{P})$: $\mathcal{O}(\sqrt{d}n^{-\frac{1}{2}}\sqrt{\log n})$ error
 - Reduced $\mathcal{O}(n \log^3 n)$ time and $\mathcal{O}(\sqrt{nd} \log n)$ space with Compress++